

TEXT-BASED CHESS GAME

Code Documentation & Technical Report

C++ Console Application

Object-Oriented Design | Polymorphism | Console Rendering

25F-0798 MUHAMMAD HUZAIFA

25F-0683 MUHAMMAD UZAIR

25F-0686 MUHAMMAD AMMAR

1. Project Overview

This project is a fully functional two-player text-based chess game implemented in C++ using Object-Oriented Programming (OOP) principles. The game runs entirely in the Windows console and renders the chessboard using Unicode chess symbols and ANSI escape codes for colored output.

Key highlights:

- Complete chess rule implementation including check, checkmate, and stalemate detection
- ANSI terminal colors for alternating light/dark squares and danger highlighting
- Unicode piece rendering for an aesthetic console display
- Captured piece tracking displayed below the board
- Move safety validation to prevent the player from leaving their king in check

2. Object-Oriented Design

2.1 Class Hierarchy

The application is built around a clean inheritance hierarchy centered on the abstract Piece base class:

Class	Type	Role
Piece	Abstract Base	Defines interface: getSymbol(), getColor(), virtual valid()
Pawn	Derived	Forward movement, diagonal capture, double first-move
Rook	Derived	Horizontal and vertical sliding movement
Knight	Derived	L-shaped jumps (2+1 squares)
Bishop	Derived	Diagonal sliding movement
Queen	Derived	Combined rook + bishop movement
King	Derived	One-square movement in any direction

Class	Type	Role
Board	Composite	Owns the 8x8 grid, game loop, rendering, and logic

2.2 OOP Concepts Applied

OOP Concept	How It Is Used
Abstraction	Piece is abstract with pure virtual valid() forcing subclasses to define movement
Inheritance	All six piece types publicly inherit from Piece
Polymorphism	Board calls piece->valid() dynamically; correct subclass method dispatched at runtime
Composition	Board composes a 8x8 array of Piece pointers (has-a relationship)
Encapsulation	Color and symbol are private in Piece; accessed through getColor() / getSymbol()

3. Architecture & File Structure

The entire project resides in a single .cpp file with the following logical organization:

Section	Contents
System Setup	enableANSI(), SetConsoleOutputCP(), constants (rows, cols)
Color Enum	WHITE and BLACK enum values
Piece Class	Abstract base with color, symbol, getters, pure virtual valid()
Piece Subclasses (headers)	Forward declarations: Pawn, Rook, Knight, Bishop, Queen, King
Board Class	Grid, captured arrays, all game logic methods
Piece::valid() Definitions	Full move validation logic for each piece type
main()	ANSI/UTF-8 setup, Board instantiation, startScreen(), start()

4. Key Components

4.1 Board Initialization

The Board constructor calls `initializeEmpty()` (sets all 64 pointers to `nullptr`) then `setupPiece()` which places all 32 pieces (16 white, 16 black) on their standard chess starting positions using new `Piece(Color)` allocations.

4.2 Move Validation

Every move goes through a multi-step validation pipeline:

Step	Check	Method
1	Input format (e.g. e2 e4)	<code>validMove(string)</code>
2	Source square not empty	<code>getPiece()</code>
3	Source piece belongs to current player	<code>getColor()</code> comparison
4	Piece-specific movement rules	<code>piece->valid()</code> (polymorphic)
5	Destination not occupied by own piece	<code>getColor()</code> on destination
6	Move does not leave own king in check	<code>isMoveSafe()</code>

4.3 Check & Checkmate Detection

- `isUnderAttack(row, col, attacker)`: iterates all attacker pieces and calls `valid()` to test if any can reach the target square.
- `isMoveSafe()`: simulates a move, calls `isUnderAttack()` on the king's position, then restores the board state.
- `hasValidMoves(player)`: iterates all player pieces and all 64 target squares; returns false only when no legal move exists (triggering checkmate or stalemate).

4.4 Piece Movement Logic

Piece	Movement Rules Implemented
Pawn	1 forward; 2 forward from starting rank; diagonal capture only; blocked by obstacles

Piece	Movement Rules Implemented
Rook	Any number of squares horizontally or vertically; blocked by intervening pieces
Knight	Exactly 2+1 or 1+2 squares (L-shape); jumps over other pieces
Bishop	Any number of squares diagonally; blocked by intervening pieces
Queen	Combines Rook + Bishop logic in a single valid() method
King	Exactly 1 square in any direction; self-check prevention via isMoveSafe

4.5 Rendering System

The display() method renders the board using:

- Unicode box-drawing characters for grid lines (┌┐└┘├┤┞┟┠┡┢┣┤┥┦┧┨┩┪┫┬┭┮┯┰┱┲┳┴┵┶┷┸┹┺┻┼┽┾┿┺┻┼┽┾┿)
- ANSI 256-color escape codes: light squares (color 247), dark squares (color 239), danger highlight (color 196 red for checked king)
- Unicode chess piece symbols: White pieces ♔♕♖♗♘♙, Black pieces ♚♛♜♝♞♟
- Captured piece display using getUnicodePiece() mapped from symbol characters

5. Game Loop Flow

The start() method drives the game with the following cycle:

Step	Action
1. Determine turn	isWhite flag alternates each round; currentPlayer and opponent derived
2. Check detection	findKing() locates king; isUnderAttack() tests if in check
3. Render board	display() called with/without danger highlight based on check state
4. Accept input	Reads two tokens (from, to) from stdin
5. Validate move	Full pipeline (Steps 1-6 from Section 4.2)
6. Execute move	setPiece() updates grid; captured piece stored and deleted
7. Post-move logic	hasValidMoves() determines checkmate or stalemate; else toggle turn

6. Known Limitations

The following standard chess rules are not implemented in this version:

Feature	Status	Notes
Castling	Not implemented	King/Rook special first-move rule
En Passant	Not implemented	Special pawn capture after double-step
Pawn Promotion	Not implemented	No promotion when pawn reaches back rank
Move Timer	Not implemented	No clock or time control
Draw by Repetition	Not implemented	No threefold-repetition tracking
Windows Only	Platform limitation	Uses windows.h; not portable to Linux/macOS

7. Memory Management

Dynamic memory is managed manually with raw pointers:

- Each Piece is heap-allocated with `new` in `setupPiece()` and in-game (if promoted, though not implemented).
- Captured pieces are explicitly deleted after being removed from the grid.
- The Board destructor iterates the full 8x8 grid and deletes any non-null Piece pointers, preventing leaks.

Note: Use of raw pointers is intentional for educational demonstration of OOP and manual resource management. A production version could use `std::unique_ptr` for safer ownership.

8. Summary

This chess game successfully demonstrates:

Area	Achievement
OOP Design	Clean class hierarchy with abstract base, 6 concrete piece types, and a composite Board
Polymorphism	Runtime dispatch of valid() across all piece types through Piece pointers
Game Logic	Full check, checkmate, and stalemate detection with move-safety simulation
Console UI	ANSI-colored, Unicode-rendered chessboard with captured piece tracking
Input Handling	Algebraic notation input (e.g., e2 e4) with full bounds and turn validation

The project is a solid foundation for extending to include advanced chess rules, an AI opponent, or a cross-platform GUI frontend.